

**LAPORAN PRAKTIKUM**  
**STRUKTUR DATA**  
Double Linked List

disusun Oleh:

**MUHAMAD FAJRI AULIA**  
NIM 2511532009

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T



FAKULTAS TEKNOLOGI INFORMASI  
DEPARTEMEN INFORMATIKA  
UNIVERSITAS ANDALAS  
PADANG, 12 MEI 2026

## KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan keenam dengan judul “**Double Linked List**”. Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai Double Linked List dengan java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 7 Mei 2026

Muhamad Fajri Aulia

# DAFTAR ISI

|                                        |     |
|----------------------------------------|-----|
| LAPORAN PRAKTIKUM .....                | 1   |
| KATA PENGANTAR.....                    | i   |
| DAFTAR ISI.....                        | ii  |
| DAFTAR GAMBAR.....                     | iii |
| BAB I PENDAHULUAN .....                | 1   |
| 1.1 Latar Belakang.....                | 1   |
| 1.2 Tujuan Praktikum .....             | 1   |
| 1.3 Persyaratan Praktikum.....         | 1   |
| 1.4 Waktu dan tempat pelaksanaan ..... | 2   |
| BAB II PEMBAHASAN.....                 | 3   |
| 2.1 Pengertian Stack .....             | 3   |
| 2.2 Langkah Pengerjaan .....           | 3   |
| 2.2.1 Node DLL .....                   | 3   |
| 2.2.2 InsertDLL .....                  | 4   |
| 2.2.3 PenelusuranDLL.....              | 6   |
| 2.2.4 HapusDLL.....                    | 8   |
| BAB III KESIMPULAN DAN SARAN .....     | 11  |
| 3.1 Kesimpulan.....                    | 11  |
| BAB IV LAPORAN TUGAS .....             | 12  |
| 4.1 Kode Program .....                 | 12  |
| 4.1.1 Lagu .....                       | 12  |
| 4.1.2 Musik.....                       | 12  |
| 4.2 Output yang dihasilkan .....       | 15  |
| DAFTAR PUSTAKA .....                   | 18  |

## DAFTAR GAMBAR

|               |    |
|---------------|----|
| Gambar 1..... | 6  |
| Gambar 2..... | 8  |
| Gambar 3..... | 10 |
| Gambar 4..... | 15 |
| Gambar 5..... | 16 |
| Gambar 6..... | 17 |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Struktur data merupakan bagian penting dalam pemrograman karena digunakan untuk mengatur dan menyimpan data secara efisien. Salah satu struktur data linear yang sering digunakan adalah Linked List. Linked List memiliki beberapa jenis, salah satunya yaitu Double Linked List.

Double Linked List adalah struktur data yang setiap node memiliki dua pointer, yaitu pointer next untuk menunjuk node berikutnya dan pointer prev untuk menunjuk node sebelumnya. Dengan adanya dua arah pointer tersebut, proses penelusuran data dapat dilakukan dari depan maupun belakang.

Pada praktikum ini digunakan bahasa pemrograman Java untuk mengimplementasikan operasi dasar Double Linked List seperti insert, traversal, dan delete. Praktikum ini membantu mahasiswa memahami cara kerja hubungan antar node serta pengelolaan pointer pada struktur data dinamis.

### **1.2 Tujuan Praktikum**

- Memahami konsep dasar Double Linked List.
- Memahami struktur node pada Double Linked List.
- Mempelajari proses penambahan node pada awal, akhir, dan posisi tertentu.

### **1.3 Persyaratan Praktikum**

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut :

- Telah mengikuti perkuliahan teori Struktur data sebagai dasar pemahaman.
- Membawa perlengkapan yang diperlukan, antara lain laptop atau computer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Enviroment (IDE) yang direkomendasikan.

- Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
- Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
- Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

#### **1.4 Waktu dan tempat pelaksanaan**

Pelaksanakan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing masing, selama memenuhi syarat teknis yang ditetapkan.

## BAB II PEMBAHASAN

### 2.1 Pengertian Stack

Double Linked List adalah linked list yang setiap nodenya memiliki dua pointer dimana strukturnya adalah dimulai dan diakhir dengan nilai null dan di dalamnya ada data node node. Double Linked List memiliki navigasi 2 arah yang diberikan dengan adanya pointer next untuk mengarah ke node selanjutnya dan juga pointer prev untuk mengarah ke node sebelumnya.

Kelebihan double linked list adalah bisa bergerak maju dan mundur dan bisa melakukan penghapusan node dengan lebih mudah. Tetapi, double linked list akan membutuhkan memori lebih besar dan memiliki struktur lebih kompleks. Penggunaan double linked list biasanya digunakan dalam fitur undo ataupun redo pada aplikasi. Navigasi halaman pada browser yang memungkinkan kita untuk melihat history halaman browse yang sudah kita buka. Serta sistem playlist lagu yang memungkinkan kita untuk melihat urutan playlist dari awal ataupun dari akhir playlist..

### 2.2 Langkah Pengerjaan

#### 2.2.1 Node DLL

Kode program :

```
package pekan6_2511532009;

public class NodeDLL_2511532009 {
    // mendefinisikan kelas node
    int data_2009; // data
    NodeDLL_2511532009 next_2009; // pointer ke next node
    NodeDLL_2511532009 prev_2009; // Pointer ke previous node

    // konstruktor
    public NodeDLL_2511532009 (int data) {
        this.data_2009 = data;
        this.next_2009 = null;
        this.prev_2009 = null;
    }
}
```

Penjelasan :

Pada program berikut di buat sebuah class dengan nama NodeDLL untuk membuat struktur node pada double linked list yang mana node merupakan

elemen utama yang menyimpan data dan hubungan antar kode. Disini kita membuat variabel data untuk menyimpan data pada mode kemudian node next untuk menunjukkan node berikutnya kemudian node prev untuk menunjukkan node sebelumnya kemudian kita membuat sebuah konstruktor yaitu public node dll untuk membuat node baru dengan nilai data tertentu Dimana this data = data lalu kita membuat mengisi data sesuai nilai dengan menyatakan bahwa sebelum dan sesudah adalah null atau masih kosong.

### 2.2.2 InsertDLL

Kode program :

```
package pekan6_2511532009;
public class InsertDLL_2511532009 {
    //menambahkan node di awal kiri
    static NodeDLL_2511532009 InsertBegin_2009 (NodeDLL_2511532009 head_2009, int
data_2009) {
        //buat node baru
        NodeDLL_2511532009 new_node_2009 = new NodeDLL_2511532009 (data_2009);
        //jadikan pointer nextnya head
        new_node_2009.next_2009 = head_2009;
        // jadikan pointer prev head ke new_node
        if (head_2009 != null) {
            head_2009.prev_2009 = new_node_2009;
        }
        return new_node_2009;
    }

    //fungsi menambahkan node di akhir
    public static NodeDLL_2511532009 insertEnd_2009 (NodeDLL_2511532009 head_2009,
int newData_2009) {
        //buat node baru
        NodeDLL_2511532009 newNode_2009 = new NodeDLL_2511532009 (newData_2009);
        //jika dll null jadikan head
        if(head_2009 == null) {
            head_2009 = newNode_2009;
        }
        else {
            NodeDLL_2511532009 curr_2009 = head_2009;
            while (curr_2009.next_2009 != null) {
                curr_2009 = curr_2009.next_2009;
            }
            curr_2009.next_2009 = newNode_2009;
            newNode_2009.prev_2009 = curr_2009;
        }
        return head_2009;
    }

    //fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511532009 insertAtPosition_2009 (NodeDLL_2511532009
head_2009, int pos_2009, int new_data_2009) {
        //buat node baru
```



```

NodeDLL_2511532009 new_node_2009 = new NodeDLL_2511532009 (new_data_2009);
if (pos_2009== 1) {
    new_node_2009.next_2009 = head_2009;
    if (head_2009 != null) {
        head_2009.prev_2009 = new_node_2009;}
    head_2009 = new_node_2009;
    return head_2009;
}

NodeDLL_2511532009 curr_2009 = head_2009;
for ( int i = 1; i <pos_2009 -1 && curr_2009 != null; i++) {
    curr_2009 = curr_2009.next_2009;}
if(curr_2009 == null) {
    System.out.println("Posisi tidak ada");
    return head_2009;
}

new_node_2009.prev_2009 = curr_2009;
new_node_2009.next_2009 = curr_2009.next_2009;
curr_2009.next_2009 = new_node_2009;
if (new_node_2009.next_2009 != null) {
    new_node_2009.next_2009.prev_2009 = new_node_2009;
}
return head_2009;
}

public static void printList_2009 (NodeDLL_2511532009 head_2009) {
    NodeDLL_2511532009 curr_2009 = head_2009;
    while (curr_2009 != null) {
        System.out.print(curr_2009.data_2009 + " <-> ");
        curr_2009 = curr_2009.next_2009;
    }
    System.out.println();
}

public static void main (String [] args) {
    //membuat dll 2 <-> 1 <-> 5
    NodeDLL_2511532009 head_2009 = new NodeDLL_2511532009(2);
    head_2009.next_2009 = new NodeDLL_2511532009(3);
    head_2009.next_2009.prev_2009 = head_2009;
    head_2009.next_2009.next_2009 = new NodeDLL_2511532009 (5);
    head_2009.next_2009.next_2009.prev_2009 = head_2009.next_2009;
    //cetak DLL awal
    System.out.print("DLL Awal : ");
    printList_2009 (head_2009);
    //tambah 1 di awal
    head_2009 = InsertBegin_2009 (head_2009,1);
    System.out.print("simpul 1 ditambah di awal: ");
    printList_2009(head_2009);
    //tambah 6 di akhir
    System.out.print("simpul 6 ditambah di akhir: ");
    int data_2009 = 6;
    head_2009 = insertEnd_2009 (head_2009, data_2009);
    printList_2009(head_2009);
    //menambah node 4 di posisi 4
    System.out.print("tambah node 4 di posisi 4: ");
    int data2_2009 = 4;
    int pos_2009 = 4;
    head_2009 = insertAtPosition_2009 (head_2009, pos_2009, data2_2009);
    printList_2009(head_2009);
}

```

```
}
}
```

Penjelasan :

Pada program ini kita membuat beberapa fungsi. Pertama fungsi insert begin Fungsi ini digunakan untuk menambahkan node di awal linked list.lalu kita membuat sebuah node baru lalu di arahkan node baru tadi ke head lama atau head baru dengan menggunakan pointer lalu node baru akan menjadi head baru. Kedua yaitu fungsi insert end untuk menambah node diakhir linked list Dimana kita akan melakukan traversal sampai node terakhir lalu dihubungkan ke node baru.lalu pointer prev node baru diarahkan ke node sebelumnya. Lalu fungsi insert at position untuk menambahkan node pada posisi tertentu Dimana transversal menuju posisi sebelum Lokasi insert lalu pointer prev node baru diarahkan ke node sebelumnya lalu next node baru diarahkan ke node setelahnya lalu node sebelumnya dihubungkan ke node baru. Lalu fungsi print list untuk mencetak isi dari double linked list Dimana transversal dilakukan selama node belum null.

Output yang dihasilkan adalah sebagai berikut:

```
DLL Awal : 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Gambar 1

### 2.2.3 PenelusuranDLL

Kode program :

```
package pekan6_2511532009;

public class PenelusuranDLL_2511532009 {
    // fungsi penelusuran maju
    static void forwardTraversal (NodeDLL_2511532009 head_2009) {
        // memulai penelusuran dari head
```

```

        NodeDLL_2511532009 curr = head_2009;
        // lanjutkan sampai akhir
        while (curr != null) {
            // print data
            System.out.print(curr.data_2009 + " <-> ");
            // pindah ke node berikutnya
            curr = curr.next_2009;
        }
        // print spasi
        System.out.println();
    }
    // fungsi penelusuran mundur
    static void backwardTransversal (NodeDLL_2511532009 tail_2009) {
        // mulai dari akhir
        NodeDLL_2511532009 curr = tail_2009;
        // lanjut sampai head
        while (curr != null) {
            // cetak data
            System.out.print(curr.data_2009 + " <-> ");
            // pindah ke node sebelumnya
            curr = curr.prev_2009;
        }
        // cetak spasi
        System.out.println();
    }
    public static void main (String[] args) {
        // cetak DLL
        NodeDLL_2511532009 head_2009 = new NodeDLL_2511532009(1);
        NodeDLL_2511532009 second_2009 = new NodeDLL_2511532009(2);
        NodeDLL_2511532009 third_2009 = new NodeDLL_2511532009(3);

        head_2009.next_2009 = second_2009;
        second_2009.prev_2009 = head_2009;
        second_2009.next_2009 = third_2009;
        third_2009.prev_2009 = second_2009;

        System.out.println("Penelusuran maju: ");
        forwardTraversal (head_2009);

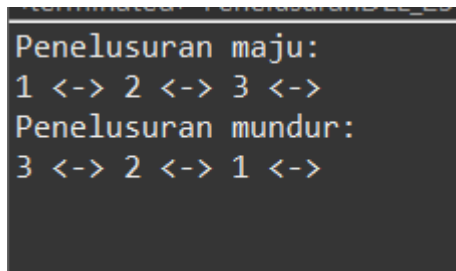
        System.out.println("Penelusuran mundur: ");
        backwardTransversal(third_2009);
    }
}

```

Penjelasan :

Paa program ini untuk melakukan traversal maju mundur pada double linked list Dimana fungsi forward untuk melakukan penelusuruan dari head menuju tail lalu pointer bergerak ke node berikutnya lalu berhenti Ketika curr bernilai null. Lalu ada fungsi untuk melakukan penelusuran menuju head dengan backward Dimana pointer bergerak ke node sebelumnya transeversal juga akan berhenti Ketika curr bernilai null.

Output yang dihasilkan adalah sebagai berikut :



```
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

Gambar 2

## 2.2.4 HapusDLL

Kode Program :

```
package pekan6_2511532009;
public class HapusDLL_2511532009 {
    //fungsi menghapus node awal
    public static NodeDLL_2511532009 delHead_2009 (NodeDLL_2511532009 head_2009) {
        if (head_2009 == null) {
            return null;
        }
        NodeDLL_2511532009 temp_2009 = head_2009;
        head_2009 = head_2009.next_2009;
        if (head_2009 != null) {
            head_2009.prev_2009 = null;
        }
        return head_2009;
    }
    //fungsi menghapus di akhir
    public static NodeDLL_2511532009 delLast_2009 (NodeDLL_2511532009 head_2009) {
        if (head_2009 == null) {
            return null;
        }
        if (head_2009.next_2009 == null ) {
            return null;
        }
        NodeDLL_2511532009 curr_2009 = head_2009;
        while (curr_2009.next_2009 != null) {
            curr_2009 = curr_2009.next_2009;
        }
        //update pointer previous node
        if (curr_2009.prev_2009 != null) {
            curr_2009.prev_2009.next_2009 = null;
        }
        return head_2009;
    }

    //fungsi menghapus node posisi tertentu
    public static NodeDLL_2511532009 delPos_2009 (NodeDLL_2511532009 head_2009,
int pos_2009) {
```

```

//jika DLL kosong
if(head_2009 == null) {
    return head_2009;
}
NodeDLL_2511532009 curr_2009 = head_2009;
//telusuri sampai ke node yang akan dihapus
for (int i =1; curr_2009 != null && i<pos_2009; i++) {
    curr_2009 = curr_2009.next_2009;
}
//jika posisi tidak ditemukan
if (curr_2009 == null) {
    return head_2009;
}
//update pointer
if (curr_2009.prev_2009 != null) {
    curr_2009.prev_2009.next_2009 = curr_2009.next_2009;
}
if (curr_2009.next_2009 != null) {
    curr_2009.next_2009.prev_2009 = curr_2009.prev_2009;
}
//jika yang dihapus head
if (head_2009 == curr_2009) {
    head_2009 = curr_2009.next_2009;
}
return head_2009;
}
//fungsi mencetak DLL
public static void printList_2009(NodeDLL_2511532009 head_2009) {
    NodeDLL_2511532009 curr_2009 = head_2009;
    while (curr_2009 != null) {
        System.out.print(curr_2009.data_2009 + " <-> ");
        curr_2009 = curr_2009.next_2009;
    }
    System.out.println();
}

public static void main (String [] args) {
    //buat sebuah DLL
    NodeDLL_2511532009 head_2009 = new NodeDLL_2511532009 (1);
    head_2009.next_2009 = new NodeDLL_2511532009(2);
    head_2009.next_2009.prev_2009 = head_2009;
    head_2009.next_2009.next_2009 = new NodeDLL_2511532009 (3);
    head_2009.next_2009.next_2009.prev_2009 = head_2009.next_2009;
    head_2009.next_2009.next_2009.next_2009 = new NodeDLL_2511532009 (4);
    head_2009.next_2009.next_2009.next_2009.prev_2009 =
head_2009.next_2009.next_2009;
    head_2009.next_2009.next_2009.next_2009.next_2009 = new NodeDLL_2511532009
(5);
    head_2009.next_2009.next_2009.next_2009.next_2009.prev_2009 =
head_2009.next_2009.next_2009.next_2009;

    System.out.print("DLL awal : " );
    printList_2009(head_2009);

    System.out.print("Setelah head dihapus : " );
    head_2009 = delHead_2009(head_2009);
    printList_2009(head_2009);

    System.out.print("Setelah node terakhir dihapus : " );

```

```

    head_2009 = delLast_2009(head_2009);
    printList_2009(head_2009);

    System.out.print("menghapus node ke 2 : " );
    head_2009 = delPos_2009(head_2009,2);
    printList_2009(head_2009);
}
}

```

Penjelasan :

Pada program ini bertujuan untuk menghapus ode pada double linked list dengan fungsi del head untuk menghaspus node pertama lalu head dipindahkan ke node berikutnya lalu prev head baru di osongkan lalu del last untuk menghapus node terakhir lalu traversal menuju node terakhir. Lalu del pos untuk menghapus node pada posisi tertentu Dimana node sebelumnya dihubungkan ke node setelah node yangdihapus lalu dibuat fungsi print untuk mencetak hasil dari program.

Output yang dihasilkan adalah sebagai berikut :

```

DLL awal : 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus : 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus : 2 <-> 3 <-> 4 <->
menghapus node ke 2 : 2 <-> 4 <->

```

Gambar 3

## **BAB III**

### **KESIMPULAN DAN SARAN**

#### **3.1 Kesimpulan**

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Double Linked List merupakan struktur data linear yang setiap node memiliki dua pointer, yaitu next dan prev. Pointer tersebut memungkinkan proses traversal dilakukan dua arah, yaitu dari depan ke belakang maupun dari belakang ke depan.

Melalui praktikum ini dapat dipahami bahwa pengelolaan pointer next dan prev sangat penting agar hubungan antar node tetap terhubung dengan benar. Double Linked List memiliki kelebihan dalam fleksibilitas traversal dibanding Single Linked List karena dapat bergerak dua arah.

## BAB IV LAPORAN TUGAS

### 4.1 Kode Program

#### 4.1.1 Lagu

```
package pekan6_2511532009;

public class Lagu_2511532009 {
    // mendefinisikan kelas node untuk lagu
    private String judul_2009; // judul lagu
    private String penyanyi_2009; // nama penyanyi
    private Lagu_2511532009 next_2009; // pointer ke node berikutnya
    private Lagu_2511532009 prev_2009; // pointer ke node sebelumnya

    // konstruktor
    public Lagu_2511532009(String judul_2009, String penyanyi_2009) {
        this.judul_2009 = judul_2009;
        this.penyanyi_2009 = penyanyi_2009;
        this.next_2009 = null;
        this.prev_2009 = null;
    }

    // Getter
    public String getJudul_2009() { return judul_2009; }
    public String getPenyanyi_2009() { return penyanyi_2009; }
    public Lagu_2511532009 getPrev_2009() { return prev_2009; }
    public Lagu_2511532009 getNext_2009() { return next_2009; }

    // Setter
    public void setJudul_2009(String judul_2009) { this.judul_2009 = judul_2009; }
    public void setPenyanyi_2009(String penyanyi_2009) { this.penyanyi_2009 =
penyanyi_2009; }
    public void setPrev_2009(Lagu_2511532009 prev_2009) { this.prev_2009 =
prev_2009; }
    public void setNext_2009(Lagu_2511532009 next_2009) { this.next_2009 =
next_2009; }
}
```

#### 4.1.2 Musik

```
package pekan6_2511532009;

import java.util.Scanner;

public class Musik_2511532009 {
    static Scanner scanner_2009 = new Scanner(System.in); // scanner untuk input
    pengguna

    public static Lagu_2511532009 tambahLagu_2009(Lagu_2511532009 head_2009,
String judul_2009, String penyanyi_2009) {
        Lagu_2511532009 laguBaru_2009 = new Lagu_2511532009(judul_2009,
penyanyi_2009); // buat node lagu baru
        if (head_2009 == null) {
            head_2009 = laguBaru_2009; // jika kosong, jadi head
        } else {
```



```

        Lagu_2511532009 curr_2009 = head_2009;
        while (curr_2009.getNext_2009() != null) {
            curr_2009 = curr_2009.getNext_2009();
        }
        curr_2009.setNext_2009(laguBaru_2009);
        laguBaru_2009.setPrev_2009(curr_2009);
    }
    return head_2009; // kembalikan head yang sudah diupdate
}

public static Lagu_2511532009 hapusLaguAwal_2009(Lagu_2511532009 head_2009) {
    if (head_2009 == null) {
        System.out.println("Playlist kosong!");
        return null;
    }
    System.out.println("Lagu \"" + head_2009.getJudul_2009() + "\" dihapus!");
    if (head_2009.getNext_2009() != null) {
        head_2009 = head_2009.getNext_2009();
        head_2009.setPrev_2009(null);
    } else {
        head_2009 = null;
    }
    return head_2009;
}

public static void tampilMaju_2009(Lagu_2511532009 head_2009) {
    if (head_2009 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    System.out.println("\n=== Playlist (Maju) ===");
    Lagu_2511532009 curr_2009 = head_2009;
    int nomor_2009 = 1;
    while (curr_2009 != null) {
        System.out.println(nomor_2009 + ". " + curr_2009.getJudul_2009() + " - " + curr_2009.getPenyanyi_2009());
        curr_2009 = curr_2009.getNext_2009();
        nomor_2009++;
    }
    System.out.println("=====");
}

public static void tampilMundur_2009(Lagu_2511532009 head_2009) {
    if (head_2009 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511532009 curr_2009 = head_2009;
    while (curr_2009.getNext_2009() != null) {
        curr_2009 = curr_2009.getNext_2009();
    }
    System.out.println("\n=== Playlist (Mundur) ===");
    int nomor_2009 = 1;
    while (curr_2009 != null) {
        System.out.println(nomor_2009 + ". " + curr_2009.getJudul_2009() + " - " + curr_2009.getPenyanyi_2009());
        curr_2009 = curr_2009.getPrev_2009();
        nomor_2009++;
    }
}

```

```

        System.out.println("=====");
    }

    public static void cariLagu_2009(Lagu_2511532009 head_2009, String judul_2009)
    {
        if (head_2009 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511532009 curr_2009 = head_2009;
        boolean ditemukan_2009 = false;
        while (curr_2009 != null) {
            if (curr_2009.getJudul_2009().equalsIgnoreCase(judul_2009)) {
                System.out.println("Ditemukan: " + curr_2009.getJudul_2009() + " - " + curr_2009.getPenyanyi_2009());
                ditemukan_2009 = true;
                break;
            }
            curr_2009 = curr_2009.getNext_2009();
        }
        if (!ditemukan_2009) {
            System.out.println("Lagu \"" + judul_2009 + "\" tidak ditemukan.");
        }
    }

    public static void tampilkanMenu_2009() {
        Lagu_2511532009 head_2009 = null;
        int pilihan_2009;
        do {
            System.out.print("\n=== Playlist Musik NIM: 2511532009 ===\n"
                + "1. Tambah Lagu\n2. Hapus Lagu Pertama\n3. Lihat Playlist (Maju)\n"
                + "4. Lihat Playlist (Mundur)\n5. Cari Lagu\n6. Keluar\nPilihan: ");
            pilihan_2009 = scanner_2009.nextInt();
            scanner_2009.nextLine();
            switch (pilihan_2009) {
                case 1:
                    System.out.print("Judul: ");
                    String judul_2009 = scanner_2009.nextLine();
                    System.out.print("Penyanyi: ");
                    String penyanyi_2009 = scanner_2009.nextLine();
                    head_2009 = tambahLagu_2009(head_2009, judul_2009, penyanyi_2009);
                    System.out.println("Lagu berhasil ditambahkan!");
                    break;
                case 2:
                    head_2009 = hapusLaguAwal_2009(head_2009);
                    break;
                case 3:
                    tampilMaju_2009(head_2009);
                    break;
                case 4:
                    tampilMundur_2009(head_2009);
                    break;
                case 5:
                    System.out.print("Cari judul: ");
                    String cari_2009 = scanner_2009.nextLine();
                    cariLagu_2009(head_2009, cari_2009);

```

```

        break;
    case 6:
        System.out.println("Terima kasih!");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
    }
} while (pilihan_2009 != 6);
scanner_2009.close();
}

public static void main(String[] args) {
    tampilkanMenu_2009();
}
}

```

## 4.2 Output yang dihasilkan

```

=== Playlist Musik NIM: 2511532009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Fearless
Penyanyi: Taylor Swift
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Luther
Penyanyi: Kendrick Lamar
Lagu berhasil ditambahkan!

```

Gambar 4

```

=== Playlist Musik NIM: 2511532009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

=== Playlist (Maju) ===
1. Fearless - Taylor Swift
2. Luther - Kendrick Lamar
=====

=== Playlist Musik NIM: 2511532009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4

=== Playlist (Mundur) ===
1. Luther - Kendrick Lamar
2. Fearless - Taylor Swift
=====

=== Playlist Musik NIM: 2511532009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Cari judul: Luther
Ditemukan: Luther - Kendrick Lamar

```

Gambar 5

```
=== Playlist Musik NIM: 2511532009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu "Fearless" dihapus!

=== Playlist Musik NIM: 2511532009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Terima kasih!
```

Gambar 6

## **DAFTAR PUSTAKA**

- [ 1 ] Modul Praktikum Struktur Data Pekan 6
- [ 2 ] Data Structures and Algorithms in Java